

Final project for the Cloud Computing Technologies course (2025/26 academic year)

KakfaDepartures: a Real-Time Cloud-Native departure dashboard for three European airports (Amsterdam-Schiphol, Helsinki-Vantaa & Oslo-Gardermoen)

Marco Colangelo (67045A)

Università degli Studi di Milano, Dipartimento di Informatica "Giovanni degli Antoni" , 12.05.2026

1. Introduction

This project implements a Cloud-Native architecture for **real-time flight departures using Apache Kafka in KRaft mode**. The system ingests live data from three European airports - Amsterdam-Schiphol, Helsinki-Vantaa and Oslo-Gardermoen - through their REST APIs (Schiphol Public Flight API for AMS, Finavia Public Flights API for HEL, Avinor XML feed for OSL). The heterogeneous data is normalized into a canonical schema and consumed in parallel by multiple microservices:

- real-time visualization through a web dashboard with per-airport statistics;
- structured notifications of departed flights;
- a daily rolling counter of departures and average delays.

In this project, Kafka is the single source of truth for all derived state. The main focus is to guarantee three non-functional properties: **fault tolerance, load balancing and transport-layer security**.

2. System Architecture

The infrastructure follows a decoupled **microservice** pattern with all communication mediated by Apache Kafka. The cluster runs in KRaft mode with three brokers acting both as data nodes and as members of the metadata quorum.

3. System Description

3.1. Main Components

Below is a description of the main components of the system.

- **Schiphol, Finavia and Avinor Pollers:** three host-side Python adapters, one per airport. Each polls its provider every 60 seconds, maintains a sliding window of 20 imminent flights, translates the airport-specific API format (paginated JSON for Schiphol, namespaced XML for Helsinki and attribute-based XML for Oslo) into a canonical `FlightEvent` schema, and forwards events via HTTP POSTs to the API-producer.
- **API-producer:** a FastAPI service exposing a `POST /flight` endpoint with Pydantic validation. Publishes events to the `flight.telemetry` topic with `acks=all`, `enable.idempotence=True`, infinite retries, and gzip compression. A custom `ResilientProducer` wrapper detects the idempotent producer's fatal state - that may occur after prolonged broker unavailability - and transparently reinstantiates it without container restart.
- **Stats-aggregator:** a Kafka Consumer and Producer that builds rolling counters of departed flights and average delays per airport. On every boot, it seeks back to 00:00 UTC and rebuilds the day's state from the log. Snapshots are published on `flight.stats` every 10 seconds. The delay is computed through a cascading fallback: precomputed `delay_minutes` → `actual_departure` → `estimated_departure` → `observed_at_utc` (the time the poller observed the departure).
- **Notifier:** Kafka Consumer/Producer that emits a human-readable alert on `flight.alerts` whenever a flight transitions to DEPARTED, using the same delay cascade. Destination IATA codes are resolved to city names via the `airportsdata` library. An in-memory deduplication set guarantees one alert per flight.
- **Dashboard:** Flask web application with three independent Kafka consumers (one per topic) and a Server-Sent Events stream toward the browser. Renders three independent boards (one per airport), each with departure statistics, a scrollable feed of today's notifications, and the next 20 scheduled flights.

3.2. Streaming processing and distributed coordination

The backbone is a **three-broker Kafka cluster in KRaft mode**, eliminating the need for ZooKeeper. All metadata (broker registration, leader election, partition assignment) is managed by the internal quorum of the brokers themselves. Three topics back the dataflow:

- `flight.telemetry` : 6 partitions, replication factor = 3 and `min.insync.replicas` = 2
- `flight.stats` : 3 partitions, same replication settings as telemetry
- `flight.alerts` : 3 partitions, same replication settings as telemetry

Messages are keyed on `airport:flight_code` so that all events of the same flight land on the same partition, preserving per-flight ordering.

3.3. Containerization

All internal services (brokers, kafka-ui, API-producer, stats-aggregator, notifier and dashboard) are packaged as **Docker containers** and orchestrated via **Docker Compose**. The PKI is bootstrapped by a shell script that generates a self-signed root CA (`FlightFlowCA`), PKCS12 keystores for each broker, and PEM key-cert pairs for internal clients. Healthchecks ensure ordered startup. Pollers run as host-side processes (each in its own `.venv`), reflecting their natural role as external adapters.

4. Non-Functional Properties

- **Fault Tolerance:** replication factor = 3 and `min.insync.replicas = 2`. The KRaft quorum elects a new leader within milliseconds upon broker failure, with no event loss. Tested by stopping one broker the system is fully operational, and with two brokers writes are held in the producer queue and delivered automatically upon recovery. The `ResilientProducer` further mitigates the librdkafka idempotent fatal state that may occur after multiple prolonged outages. Consumer-side resilience is verified by killing the stats-aggregator: it rebuilds counters from the seek-to-midnight-UTC mechanism, with no manual recovery.
- **Load Balancing:** handled natively by Kafka through topic partitioning and consumer-group semantics.
 - Stats-aggregator and notifier use static Group IDs: in case of horizontal scaling, Kafka rebalances the 6 partitions of `flight.telemetry` evenly across replicas. The stats-aggregator is intentionally not horizontally scaled: aggregating global counters across partitions would require a downstream reduce stage, a complication unjustified at the project's data volume.
 - Dashboard uses a randomly generated Group ID (each instance forms its own group), implementing a broadcasting pattern suited for real-time visualization, where every replica must see every event.
- **Security:** all internal communication is encrypted via mTLS (Mutual TLS). Each broker holds a **PKCS12** keystore with its RSA certificate and a truststore with the CA certificate. Each Python microservice carries a **PEM-encoded** certificate-key pair signed by the internal CA `FlightFlowCA`. Both parties verify each other during the TLS handshake. Validation experiments with a rogue producer confirmed rejection at the transport layer of three distinct attacks (plain TCP, TLS without client cert, TLS with non-CA-signed cert). Packet capture on the broker port confirmed the wire payload is fully encrypted, with no application string readable.

5. Application Showcases